

3 장 실행계획 확인

실행순서 확인 문제

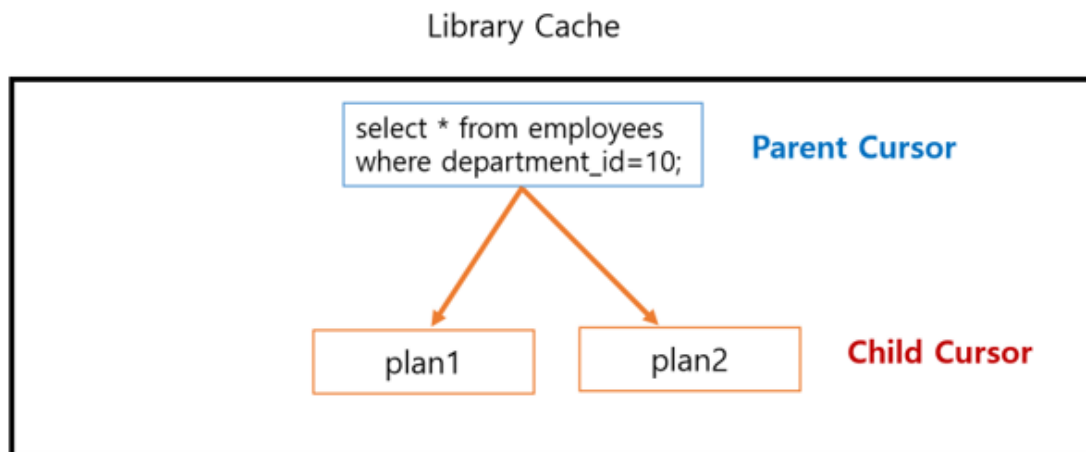
다음 실행 계획에서 수행 순서를 나열하세요.

```
SQL> EXPLAIN PLAN FOR
      SELECT  m.last_name
            ,   d.department_name
            ,   l.street_address
      FROM    hr.employees m
      JOIN    hr.departments d
      ON      d.manager_id = m.employee_id
      NATURAL JOIN hr.locations
      WHERE   l.city = 'Seattle' ;
```

```
SQL> SELECT * FROM table(DBMS_XPLAN.DISPLAY);
```

Id	Operation	Name
0	SELECT STATEMENT	
1	NESTED LOOPS	
2	NESTED LOOPS	
3	TABLE ACCESS BY INDEX ROWID	LOCATIONS
4	INDEX RANGE SCAN	LOC_CITY_IX
5	TABLE ACCESS BY INDEX ROWID	DEPARTMENTS
6	INDEX RANGE SCAN	DEPT_LOCATION_IX
7	TABLE ACCESS BY INDEX ROWID	EMPLOYEES
8	INDEX RANGE SCAN	EMP_EMP_ID_PK

Parent Cursor / Child Cursor



Parent Cursor : SQL Text 저장

Child Cursor : 수행 계획, 바인드 변수, 오브젝트 정보, 옵티마이저 환경 정보등 저장

```
SQL> conn test/oracle
Connected.
```

```
-- OPTIMIZER_MODE : all_rows
```

```
SQL> select * from hr.employees where department_id=10;
```

```
SQL> select sql_id, sql_text from v$sql
       where sql_text like '%hr.employees%';
```

```
SQL_ID
```

```
SQL_TEXT
```

```
f2ss5zna18342
select * from hr.employees where department_id=10
```

```
fq57yxn3z59mg
SELECT e.last_name, d.department_name FROM hr.employees e, hr.departments d WHERE e.department_id
=d.department_id
```

```
1650wqrnazzrp
select sql_id, sql_text from v$sql where sql_text like '%hr.employees%'
```

```
-- OPTIMIZER_MODE : first_rows_1
```

```
SQL> alter session set optimizer_mode=first_rows_1;
```

Session altered.

```
SQL> select * from hr.employees where department_id=10;
```

```
SQL> select sql_id, sql_text, child_number from v$sql
       where sql_id='f2ss5zna18342';
```

```
SQL_ID
```

```
SQL_TEXT
```

```
CHILD_NUMBER
```

```
f2ss5zna18342
```

```
select * from hr.employees where department_id=10
      0
```

```
f2ss5zna18342
```

```
select * from hr.employees where department_id=10
      1
```

```
SQL> select sql_text, version_count
       from v$sqlarea
       where sql_id='f2ss5zna18342';
```

```
SQL_TEXT
```

```
VERSION_COUNT
```

```
select * from hr.employees where department_id=10                2    <- child cursor 개수
```

```
SQL> select sql_id, child_number, OPTIMIZER_MODE_MISMATCH
       from v$sql_shared_cursor
       where sql_id='f2ss5zna18342';
```

```
SQL_ID          CHILD_NUMBER 0
```

```
f2ss5zna18342          0 N
```

```
f2ss5zna18342          1 Y
```

```
SQL> set linesize 300
```

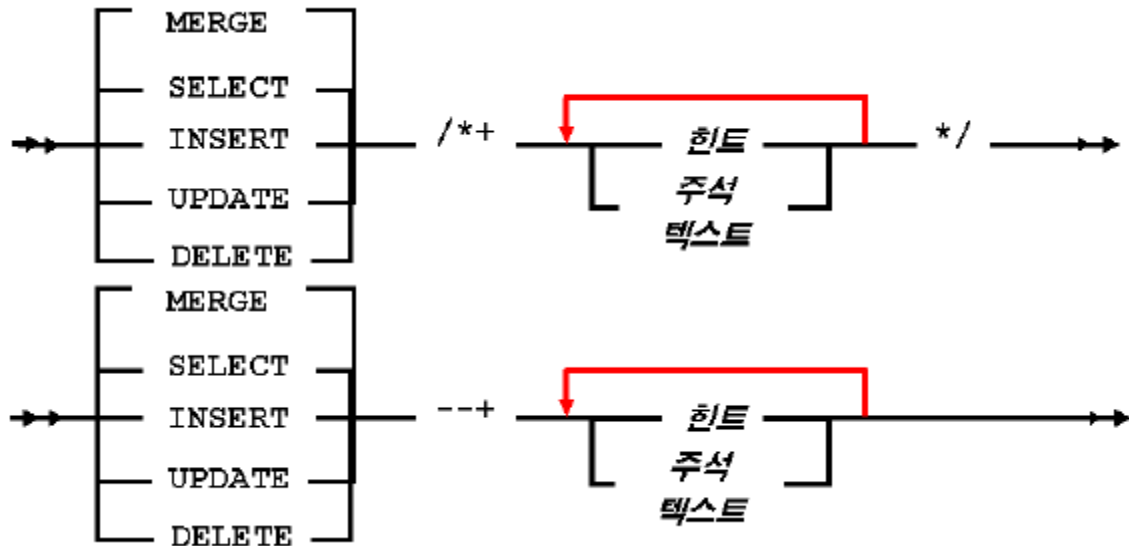
```
SQL> select sql_id, child_number, OPTIMIZER_MODE_MISMATCH, reason
       from v$sql_shared_cursor
       where sql_id='f2ss5zna18342';
```

```
SQL_ID          CHILD_NUMBER 0 REASON
```

```
f2ss5zna18342          0      N      <ChildNode><ChildNumber>0</ChildNumber><ID>3</ID><reason>Optimizer
mismatch(10)<
f2ss5zna18342          1 Y
```

옵티마이저 힌트

- 옵티마이저 결정에 영향
- 최후의 수단으로만 사용해야 함(높은 유지 관리 비용 발생)
- syntax



- 규칙
 - SQL 명령의 첫 키워드 바로 뒤에 위치
 - 힌트는 여러 개를 지정 가능
 - 해당 명령어 블록에만 적용
 - 명령어에서 별칭을 사용하는 경우는 반드시 별칭으로 힌트에 사용
 - 잘못된 힌트는 오류를 발생하지 않고 무시됨

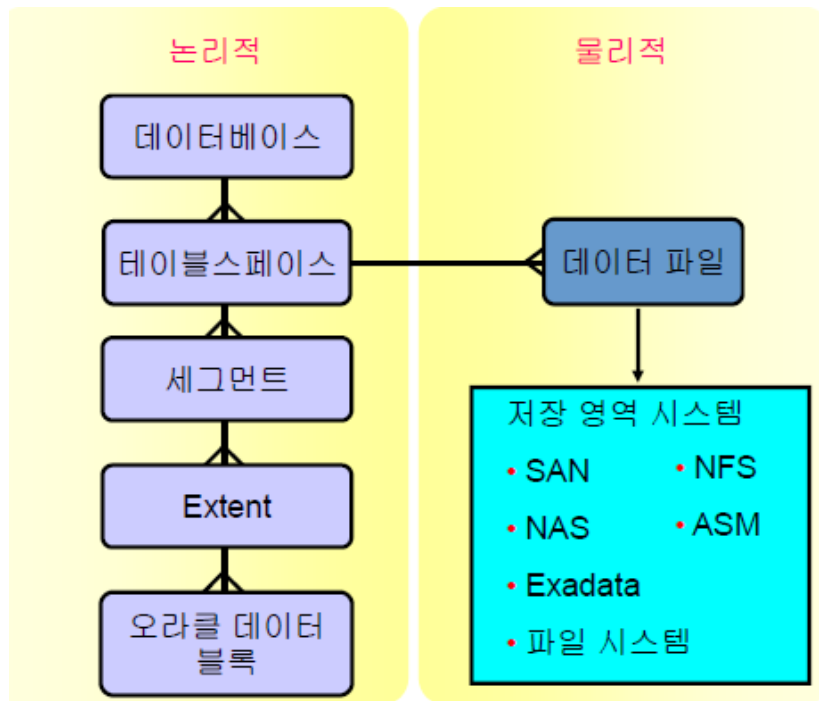
- 예제

```
SELECT /*+ INDEX(e emp_idx) */ *
FROM   employees e
WHERE  employee_id IN (SELECT /*+ INDEX(departments dept_idx) */ employee_id
                       FROM   departments
                       WHERE  department_id = 10);
```

```
SELECT /*+ QB_NAME(MAIN) INDEX(e@MAIN emp_idx)
        INDEX(departments@INLINE dept_idx) */ *
FROM   employees e
WHERE  employee_id IN ( SELECT /*+ QB_NAME(INLINE) */ employee_id
                       FROM   departments
                       WHERE  department_id = 10);
```

4 장 인덱스 튜닝

저장 영역 : 논리적 및 물리적 데이터베이스 구조



1. 데이터베이스, 테이블스페이스 및 데이터 파일

데이터베이스는 논리적으로 두 개 이상의 테이블스페이스로 나뉩니다. 테이블스페이스에 있는 모든 세그먼트의 데이터를 물리적으로 저장할 수 있도록 하나 이상의 데이터 파일이 명시적으로 생성됩니다.

2. 테이블스페이스

테이블스페이스는 관련된 논리적 구조를 그룹화합니다. 그룹화된 논리적 구조들에 대한 관리 작업을 테이블스페이스 레벨에서 수행할 수 있습니다.

3. 세그먼트

세그먼트는 특정 논리적 구조에 할당된 Extent들입니다.

- 데이터 세그먼트
- 인덱스 세그먼트
- 언두 세그먼트
- 임시 세그먼트

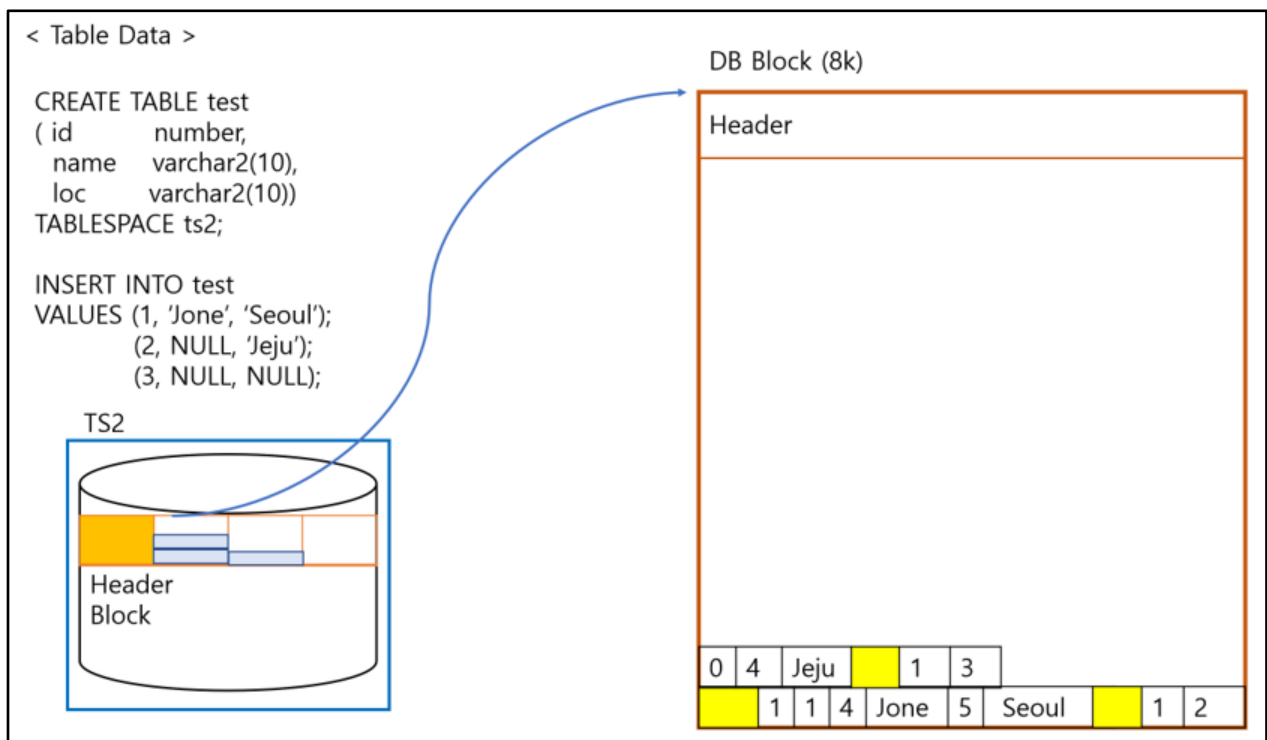
4. 익스텐트

익스텐트는 단일 할당으로 얻은 일정 수의 연속적인 오라클 데이터 블록으로, 특정 유형의 정보를 저장하는 데 사용됩니다. 익스텐트에 있는 오라클 데이터 블록들은 논리적으로 연속되어 있지만 RAID 스트라이핑 등의 구현으로 인해 물리적으로는 디스크에 분산되어 있을 수 있습니다.

5. 오라클 데이터 블록

가장 세분화된 레벨에서 오라클 데이터베이스의 데이터는 데이터 블록에 저장됩니다. 하나의 데이터 블록은 디스크에서 특정 바이트 수의 물리적 공간에 해당합니다.

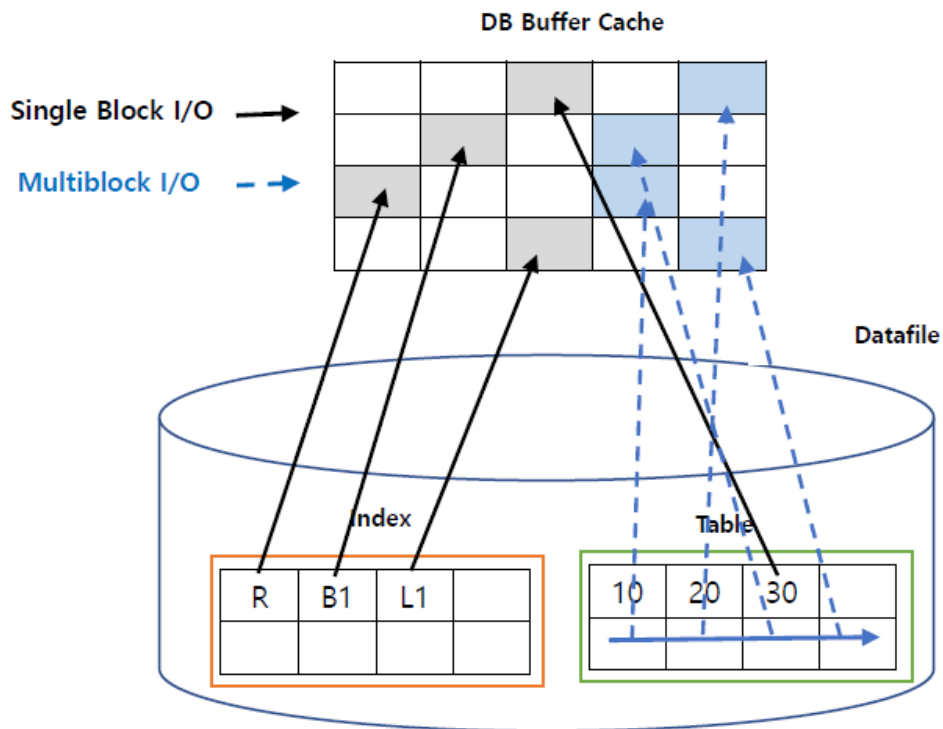
저장 영역 : 데이터가 저장되는 방식



데이터베이스는 관련된 논리적 구조를 그룹화하는 데 사용할 수 있는 테이블스페이스라는 논리적 저장 영역 단위로 나뉩니다. 각 테이블스페이스에는 테이블스페이스에 있는 모든 논리적 구조 데이터를 물리적으로 저장할 수 있는 하나 이상의 데이터 파일이 명시적으로 생성됩니다.

위의 그림은 하나의 데이터 파일로 구성된 테이블스페이스 TS2를 보여 줍니다. TEST 테이블에 첫 행을 입력하면 TS2 테이블스페이스에서 첫 익스텐트를 할당 받습니다. 익스텐트는 연속된 8KB 오라클 블록들로 이루어져 있습니다. 입력한 행들은 오라클 블록에 행 형식에 따라 저장됩니다.

Single Block I/O 와 Multiblock I/O



1. Single Block I/O

- 한 번에 한 블록씩 버퍼 캐시에 적재합니다.
- 인덱스를 사용한 검색시 이용합니다.

2. Multiblock I/O

- 물리적으로 인접한 블록 여러 개를 한 번에 버퍼 캐시에 적재합니다.
- 테이블 전체를 스캔해야 할 때 이용합니다.

3. Multiblock I/O 시 한번에 읽어내는 최대 블록 수 확인

```
SQL> show parameter db_file_multiblock_read_count
```

NAME	TYPE	VALUE
db_file_multiblock_read_count	integer	128

정렬 작업과 PGA, 임시 테이블스페이스

1. PGA(Program Global Area)

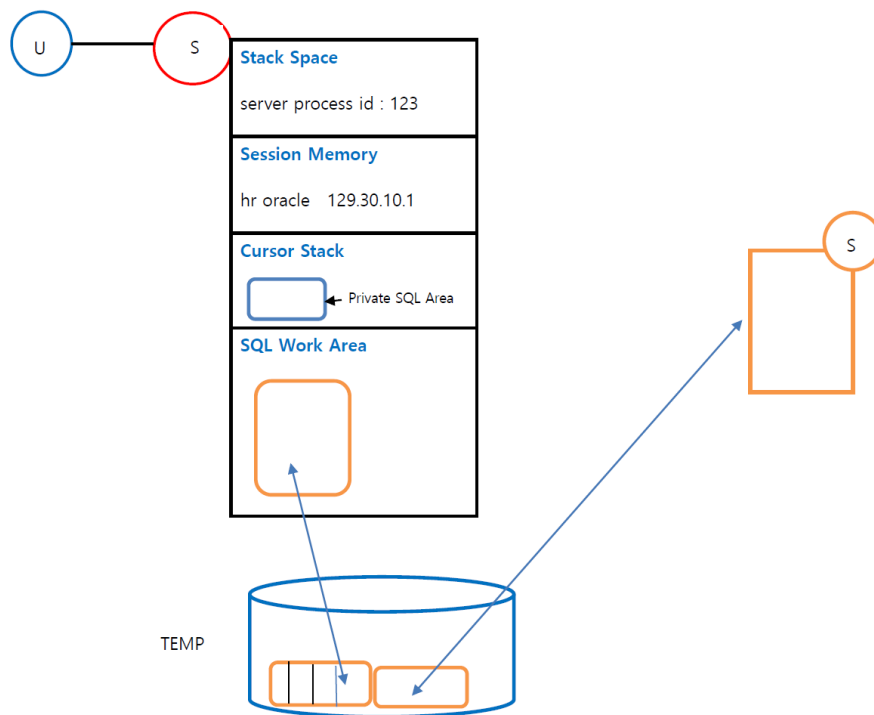
- 오라클 서버 프로세스 전용 메모리
- SQL Work Area의 Sort Area에서 정렬 작업을 수행

2. 임시 테이블스페이스

- 정렬 작업에 Sort Area가 부족할 경우 사용

3. 정렬 작업 종류

- 메모리 소트 : 정렬 작업을 메모리만으로 완료
- 디스크 소트 : 임시 테이블스페이스까지 사용하여 정렬 완료



Index Tuning

INDEX CASE 1

다음의 SELECT 문장을 실행하고 실행계획을 확인한 후 문제점을 진단하고 해결책을 제시합니다.

```
SQL> select /*+ index(emp2 emp2_x01) */ *  
      from emp2  
      where deptno=30 and sal >=2000;
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		8192	00:00:00.33	2338
* 1	TABLE ACCESS BY INDEX ROWID BATCHED	EMP2	1	30492	8192	00:00:00.33	2338
* 2	INDEX RANGE SCAN	EMP2_X01	1	38229	49152	00:00:00.14	238

Predicate Information (identified by operation id):

- 1 - filter("SAL">=2000)
- 2 - access("DEPTNO"=30)

INDEX CASE 2

다음 문장의 실행 계획을 통해 문제점을 식별하고 성능을 향상시키시오.
필요한 인덱스를 생성하시요.

```
SQL> SELECT    cust_id, cust_last_name, cust_year_of_birth, cust_city, country_id
      FROM      custs c
      WHERE     cust_credit_limit BETWEEN 1500 AND 3000
      AND       cust_year_of_birth = 1990 ;
```

```
SQL> @xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		3	00:00:00.01	1141
* 1	TABLE ACCESS FULL	CUSTS	1	175	3	00:00:00.01	1141

Predicate Information (identified by operation id):

```
1 - filter(("CUST_YEAR_OF_BIRTH"=1990 AND "CUST_CREDIT_LIMIT"<=3000 AND
          "CUST_CREDIT_LIMIT">=1500))
```

해결 1) 필터링 조건이 좋은 단일 컬럼의 인덱스를 이용

해결 2) 결합 인덱스를 이용

INDEX CASE 3

다음의 문장이 인덱스를 이용하여 정렬 작업을 대체할 수 있도록 필요한 사항을 구현하시오.

```
SQL> SELECT *  
      FROM emp  
      ORDER BY sal ASC ;
```

```
SQL> @xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		14	00:00:00.01	9			
1	SORT ORDER BY		1	14	14	00:00:00.01	9	2048	2048	2048 (0)
2	TABLE ACCESS FULL	EMP	1	14	14	00:00:00.01	9			

INDEX CASE 4

다음 문장의 성능을 확인하고 인덱스의 사용을 최적화 가능하도록 필요한 사항을 구현하시오.

```
SQL> @C:\labs\wtest\wmcustsum.sql -- mcustsum 테이블과 관련 인덱스를 생성
```

```
SQL> SELECT COUNT(*)  
      FROM mcustsum t  
      WHERE salegb = 'A'  
      AND salemm BETWEEN '200801' AND '200812' ;
```

```
SQL>@xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1	00:00:00.03	2622
1	SORT AGGREGATE		1	1	1	00:00:00.03	2622
* 2	INDEX FAST FULL SCAN	MCUST_X01	1	456K	76084	00:00:00.03	2622

Predicate Information (identified by operation id):

```
2 - filter(("SALEGB"='A' AND "SALEMM">='200801' AND "SALEMM"<='200812'))
```

INDEX CASE 5

다음 문장의 실행계획을 확인하고 인덱스를 최대한 활용할 수 있도록 필요한 사항을 구현하시오.

```
SQL> SELECT  channel_id, MAX(time_id)
        FROM    sales
        WHERE   channel_id = 3
        GROUP BY channel_id;
```

```
CHANNEL_ID MAX(TIME)
-----
3 01/12/31
```

```
SQL>@xp1an
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1	00:00:00.14	4287
1	SORT GROUP BY NOSORT		1	536K	1	00:00:00.14	4287
* 2	INDEX FAST FULL SCAN	SALES_IX01	1	536K	536K	00:00:00.07	4287

Predicate Information (identified by operation id):

2 - filter("CHANNEL_ID"=3)

5 장 조인 튜닝

조인 명령어

I. Oracle Join Syntax

```
SELECT      table1.column, table2.column
FROM        table1, table2
WHERE        table1.column1 = table2.column2;      <- 조인조건
```

1. Equijoins

```
SELECT      e.employee_id, e.last_name, e.department_id,
            d.department_id, d.location_id
FROM        employees e, departments d
WHERE       e.department_id = d.department_id;

SELECT      d.department_id, d.department_name, l.city
FROM        departments d, locations l
WHERE       d.location_id = l.location_id          <- 조인조건
AND        d.department_id IN (20, 50);           <- 비조인조건

SELECT      e.last_name, d.department_name, l.city
FROM        employees e, departments d, locations l    <- 3 tables
WHERE       e.department_id = d.department_id
AND        d.location_id = l.location_id;
```

2. Nonequijoins

```
SELECT      e.last_name, e.salary, j.grade_level
FROM        employees e, job_grades j
WHERE       e.salary BETWEEN j.lowest_sal AND j.highest_sal;

SELECT      w.last_name worker, c.last_name colleague
FROM        employees w, employees c
WHERE       w.department_id=c.department_id
AND        w.employee_id <> c.employee_id
ORDER BY    worker, colleague;
```

3. Outer joins

SELECT table1.column, table2.column

FROM table1, table2

WHERE table1.column(+) = table2.column;

```
SELECT      e.last_name, e.department_id, d.department_name
FROM        employees e, departments d
WHERE       e.department_id(+) = d.department_id ;
```

```
SELECT      e.last_name, e.department_id, d.department_name
FROM        employees e, departments d
WHERE       e.department_id = d.department_id(+);
```

4. Cartesian products

```
SELECT      last_name, department_name
FROM        employees, departments;
```

II. SQL:1999 표준 구문 조인 명령어

1. NATURAL JOIN

Natural join 은 이름과 데이터 유형이 같은 모든 열을 사용하여 테이블을 조인합니다.

Equijoin 만 가능합니다.

동일한 이름을 가진 열이 서로 다른 데이터 유형을 가지면 오류가 반환됩니다.

```
SQL> SELECT      employee_id, last_name,
                  department_id, department_name
FROM      employees e NATURAL JOIN departments d;
```

2. USING 절을 사용한 조인

여러 열의 이름이 동일하지만 데이터 유형은 다를 경우 USING 절을 사용하여 조인에 사용할 열을 Equijoin 으로 지정할 수 있습니다. 즉, USING 절을 사용하면 두 개 이상의 열이 일치하는 경우 하나의 열만 일치하도록 할 수 있습니다. NATURAL JOIN과 USING 절은 함께 사용할 수 없습니다.

USING절에 사용되는 열에는 테이블의 이름이나 테이블의 alias를 사용할 수 없습니다.

```
SQL> SELECT      employee_id, last_name,
                  department_id, department_name
FROM      employees e, departments d
USING      (department_id);
```

3. ON 절을 사용한 조인

ON 절을 사용하여 조인 조건을 지정합니다. 조인 조건에서 임의 조건을 지정하거나 조인할 열을 지정할 수 있습니다. Equijoin뿐 아니라 Nonequijoin에서도 사용 가능합니다.

```
SQL> SELECT      employee_id, last_name,
                  d.department_id, department_name
FROM      employees e JOIN departments d
ON      d.department_id = e.department_id;
```

ON 절을 사용하여 3 개 테이블 조인

```
SQL> SELECT      employee_id, first_name, city, department_name
FROM      employees e
JOIN      departments d
ON      (e.department_id = d.department_id)
JOIN      locations l
ON      (d.location_id = l.location_id);
```

ON 절을 사용하여 Non-Equi Join

```
SQL> SELECT      e.last_name, e.salary, j.grade_level
FROM      employees e
JOIN      job_grades j
ON      e.salary BETWEEN j.lowest_sal AND j.highest_sal;
```


조인에 추가 조건 적용

```
SQL> SELECT      e.employee_id, e.last_name, e.department_id,
                d.department_id, d.location_id
      FROM        employees e JOIN departments d
      ON          (e.department_id = d.department_id)
      AND         e.manager_id = 149;
```

또는

```
SQL> SELECT      e.employee_id, e.last_name, e.department_id,
                d.department_id, d.location_id
      FROM        employees e JOIN departments d
      ON          (e.department_id = d.department_id)
      WHERE       e.manager_id = 149;
```

4. OUTER JOIN : Oracle 조인 구문과 ANSI 표준 구문

- 부서가 할당되지 않는 직원까지 포함하여 모든 직원의 정보를 표시

```
SQL> SELECT      e.last_name, e.department_id, d.department_name
      FROM        employees e, departments d
      WHERE       e.department_id = d.department_id(+);
```

```
SQL> SELECT      e.last_name, e.department_id, d.department_name
      FROM        employees e LEFT OUTER JOIN departments d
      ON          e.department_id = d.department_id;
```

- 직원이 할당되지 않는 부서까지 포함하여 모든 부서의 정보를 표시

```
SQL> SELECT      e.last_name, e.department_id, d.department_name
      FROM        employees e, departments d
      WHERE       e.department_id(+) = d.department_id;
```

```
SQL> SELECT      e.last_name, e.department_id, d.department_name
      FROM        employees e RIGHT OUTER JOIN departments d
      ON          e.department_id = d.department_id;
```

- 부서가 할당되지 않은 직원까지 모든 직원과

직원이 할당되지 않은 부서까지 모든 부서를 함께 표시

```
SQL> SELECT      e.last_name, e.department_id, d.department_name
      FROM        employees e, departments d
      WHERE       e.department_id = d.department_id(+)
      UNION
      SELECT      e.last_name, e.department_id, d.department_name
      FROM        employees e, departments d
      WHERE       e.department_id(+) = d.department_id;
```

```
SQL> SELECT      e.last_name, e.department_id, d.department_name
      FROM        employees e FULL OUTER JOIN departments d
      ON          e.department_id = d.department_id;
```

5. Cartesian Product

```
SQL> SELECT      e.last_name, d.department_name  
      FROM        employees e CROSS JOIN  departments d;
```

인라인 뷰 사용예

1. 보기 좋은 Coding

```
SELECT  *
FROM    emp a, dept d
WHERE   a.deptno=b.deptno
AND     a.job=' SALESMAN'
AND     b.loc=' CHICAGO' ;
```

=>

```
SELECT  *
FROM    (SELECT * FROM emp WHERE job=' SALESMAN' ) a
,       (SELECT * FROM dept WHERE loc=' CHICAGO' ) b
WHERE   a.deptno=b.deptno;
```

2. 정렬된 결과에 rownum 을 사용해야 하는 경우

```
SELECT  ename, ROWNUM
FROM    ( SELECT ename
          FROM    emp
          ORDER BY ename);
```

3. 반복되는 표현식을 제거

```
SELECT  deptno
,       MIN(sal+NVL(comm,0)) AS sal_min
,       MAX(sal+NVL(comm,0)) AS sal_max
FROM    emp
GROUP BY deptno;
```

=>

```
SELECT  deptno
,       MIN(sal) AS sal_min
,       MAX(sal) AS sal_max
FROM    (SELECT deptno, sal+NVL(comm,0) AS sal FROM emp)
GROUP BY deptno;
```

4. 조인 순서 변경

```
SELECT e.employee_id, d.department_name, l.city
FROM employees e, departments d, locations l
WHERE e.department_id=d.department_id(+)
AND d.location_id=l.location_id(+);
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		20	00:00:00.01	7
* 1	HASH JOIN OUTER		1	20	20	00:00:00.01	7
* 2	HASH JOIN OUTER		1	20	20	00:00:00.01	4
3	TABLE ACCESS FULL	EMPLOYEES	1	20	20	00:00:00.01	2
4	TABLE ACCESS FULL	DEPARTMENTS	1	8	8	00:00:00.01	2
5	TABLE ACCESS FULL	LOCATIONS	1	5	5	00:00:00.01	3

EMPLOYEES -> DEPARTMENTS -> LOCATIONS

```
SELECT e.employee_id, dl.department_name, dl.city
FROM (SELECT d.department_id, d.department_name, l.city
      FROM departments d, locations l
      WHERE d.location_id=l.location_id) dl,
employees e
WHERE e.department_id=dl.department_id(+);
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		20	00:00:00.01	8
* 1	HASH JOIN OUTER		1	20	20	00:00:00.01	8
2	TABLE ACCESS FULL	EMPLOYEES	1	20	20	00:00:00.01	2
3	VIEW		1	8	8	00:00:00.01	6
4	MERGE JOIN		1	8	8	00:00:00.01	6
5	TABLE ACCESS BY INDEX ROWID	LOCATIONS	1	5	5	00:00:00.01	4
6	INDEX FULL SCAN	LOCATIONS_IX01	1	5	5	00:00:00.01	2
* 7	SORT JOIN		5	8	8	00:00:00.01	2
8	TABLE ACCESS FULL	DEPARTMENTS	1	8	8	00:00:00.01	2

LOCATIONS -> DEPARTMENTS -> EMPLOYEES

5. 여러 테이블의 조인과 그룹함수의 적용이 필요한 경우 그룹핑을 먼저 처리하도록 해서
조인에 참여하는 건수를 줄여 성능 향상

```
SELECT  d.department_name, AVG(salary) avg_sal, SUM(salary) sum_sal
FROM    employees e JOIN departments d
ON      e.department_id=d.department_id
GROUP BY d.department_name;
```

=>

```
SELECT  d.department_name, e.avg_sal, e.sum_sal
FROM    departments d
,        (SELECT  department_id, AVG(salary) avg_sal, SUM(salary) sum_sal
          FROM    employees
          GROUP BY department_id) e
WHERE   d.department_id=e.department_id;
```

6. 위치에 따라 참조할 수 없는 열 alias 를 참조하고자 할 때 사용

```
SELECT *
FROM ( SELECT empno, ename, sal*12 ann_sal, deptno
        FROM emp )
WHERE ann_sal >= 35000;    -- 열 alias 는 ORDER BY 절에서만 사용할 수 있습니다.
```

7. 1:1 관계의 조인을 위해 사용

EMPLOYEES(재직 직원), DEPARTMENTS, RETIRED_EMPLOYEES(퇴직 직원)테이블을 이용하여 부서별(department_name) 재직직원, 퇴직직원의 급여합을 출력합니다.

잘못 작성된 명령어

```
SELECT d.department_name,
       sum(e.salary) emp_sal, sum(r.salary) retired_sal
FROM   departments d, employees e, retired_employees r
WHERE  d.department_id=e.department_id(+)
AND    d.department_id=r.department_id(+)
GROUP BY d.department_name;
```

DEPARTMENT_NAME	EMP_SAL	RETIRED_SAL
1 Sales	30100	28500
2 Marketing	19000	(null)
3 Administration	4400	(null)
4 Contracting	(null)	(null)
5 Executive	232000	108600
6 IT	76800	69300
7 Shipping	17500	(null)
8 Accounting	20308	(null)

EMPLOYEES(m) ---- DEPARTMENTS(1) ---- RETIRED_EMPLOYEES(m)

세 테이블을 그대로 조인하면 m*m 개의 조인 결과가 만들어지는데 위의 업무는 세 테이블간의 관계에서 EMPLOYEES 테이블과 DEPARTMENTS 테이블을 조인(1*m)한 결과를 1로 만든 후 RETIRED_EMPLOYEES 와 조인(1*m) 한 것도 1의 결과가 되어야 한다.

인라인 뷰를 이용하여 EMPLOYEES, RETIRED_EMPLOYEES 테이블의 조인 대상을 1의 상태로 만들어 조인하는 것이 필요합니다.

```
SELECT d.department_name, e.emp_sal, r.retire_sal
FROM   departments d,
       (SELECT department_id, SUM(salary) emp_sal
        FROM   employees
        GROUP BY department_id) e,
       (SELECT department_id, SUM(salary) retire_sal
        FROM   retired_employees
        GROUP BY department_id) r
WHERE  d.department_id = e.department_id(+)
AND    d.department_id = r.department_id(+);
```

DEPARTMENT_NAME	EMP_SAL	RETIRE_SAL
1 Shipping	17500	(null)
2 Accounting	20308	(null)
3 Executive	58000	36200
4 Administration	4400	(null)
5 Marketing	19000	(null)
6 IT	19200	23100
7 Sales	30100	9500
8 Contracting	(null)	(null)

Join Tuning

JOIN CASE 1

다음 문장의 실행 계획을 확인하고 Join의 성능을 최적화 시킨다.

```
SQL>SELECT      c.cust_id,
                c.cust_last_name,
                c.cust_city,
                c.cust_credit_limit,
                s.prod_id,
                s.time_id,
                s.quantity_sold
FROM            custs c, sales s
WHERE           c.cust_id = s.cust_id
AND             c.cust_city = 'Los Angeles'
AND             c.cust_credit_limit > 3000
AND             s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
AND             TO_DATE('1999/12/31','YYYY/MM/DD') ;
```

SQL> @xplan

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1932	00:00:00.02	9688
1	NESTED LOOPS		1		1932	00:00:00.02	9688
2	NESTED LOOPS		1	2580	8025	00:00:00.01	2371
* 3	TABLE ACCESS FULL	CUSTS	1	80	629	00:00:00.01	1158
* 4	INDEX RANGE SCAN	SALES_IX02	629	129	8025	00:00:00.01	1213
* 5	TABLE ACCESS BY INDEX ROWID	SALES	8025	32	1932	00:00:00.01	7317

Predicate Information (identified by operation id):

```
3 - filter(("C"."CUST_CITY"='Los Angeles' AND "C"."CUST_CREDIT_LIMIT">3000))
4 - access("C"."CUST_ID"="S"."CUST_ID")
5 - filter(("S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'yyyy-mm-dd hh24:mi:ss') AND
          "S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00', 'yyyy-mm-dd hh24:mi:ss')))
```

JOIN CASE 2

다음 문장의 실행 계획을 확인하고 성능을 최적화 시킨다.

```
SQL> SELECT /*+ leading(c) use_nl(s) */ COUNT(*)
      FROM   sales s, custs c
      WHERE  s.cust_id = c.cust_id
      AND    c.country_id = 52790
      AND    s.time_id BETWEEN TO_DATE('1999/01/01','YYYY/MM/DD')
      AND    TO_DATE('1999/12/31','YYYY/MM/DD');
```

```
SQL> @xplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT		1		1	00:00:01.67	453K
1	SORT AGGREGATE		1	1	1	00:00:01.67	453K
2	NESTED LOOPS		1		140K	00:00:01.65	453K
3	NESTED LOOPS		1	94710	522K	00:00:00.37	37377
* 4	TABLE ACCESS FULL	CUSTS	1	2921	18520	00:00:00.01	1139
* 5	INDEX RANGE SCAN	SALES_IX02	18520	129	522K	00:00:00.22	36238
* 6	TABLE ACCESS BY INDEX ROWID	SALES	522K	32	140K	00:00:01.04	416K

Predicate Information (identified by operation id):

```
4 - filter("C"."COUNTRY_ID"=52790)
5 - access("S"."CUST_ID"="C"."CUST_ID")
6 - filter(("S"."TIME_ID"<=TO_DATE(' 1999-12-31 00:00:00', 'yyyy-mm-dd hh24:mi:ss') AND
          "S"."TIME_ID">=TO_DATE(' 1999-01-01 00:00:00', 'yyyy-mm-dd hh24:mi:ss')))
```


JOIN CASE 3

다음 문장의 실행 계획을 확인하고 성능을 최적화 시킨다.

```
SQL> SELECT    DISTINCT c.channel_id, c.channel_desc, s.prod_id, s.promo_id
      FROM      channels c, sales s
      WHERE     c.channel_id = s.channel_id
      AND       s.prod_id BETWEEN 10 AND 20 ;
```

```
SQL> @xpplan
```

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		53	00:00:00.02	289			
1	HASH UNIQUE		1	142	53	00:00:00.02	289	1246K	1246K	1375K (0)
* 2	HASH JOIN		1	61703	61702	00:00:00.01	289	1506K	1506K	898K (0)
3	TABLE ACCESS FULL	CHANNELS	1	5	5	00:00:00.01	9			
* 4	INDEX RANGE SCAN	SALES_IX01	1	61703	61702	00:00:00.01	280			

Predicate Information (identified by operation id):

- 2 - access("C"."CHANNEL_ID"="S"."CHANNEL_ID")
- 4 - access("S"."PROD_ID">=10 AND "S"."PROD_ID"<=20)

JOIN CASE 4

PRODS, SALES 테이블을 이용하여 제품별 판매 수량(QUANTITY_SOLD)의 합계를 다음과 같이 검색하는 명령어를 작성하세요. 판매되지 않은 제품이 존재한다면 해당 제품도 함께 표시합니다. 실행 계획을 확인하고 성능을 최적화시킵니다.

검색결과

PROD_ID	PROD_NAME	SOLD_SUM
13	5MP Telephoto Digital Camera	6002
14	17" LCD w/built-in HDTV Tuner	5998
15	Envoy 256MB - 40GB	5766
16	Y Box	6929
17	Mini DV Camcorder with 3.5" Swivel LCD	6160
18	Envoy Ambassador	9591
19	Laptop carrying case	10430
20	Home Theatre Package with DVD-Audio/Video Play	10826
...		

72 행이 선택되었습니다.